POLITECNICO
MILANO 1863

School of Civil, Environmental and Land Management Engineering

Master Degree in Civil Engineering
Academic year 2026-2027

ADVANCED COMPUTATIONAL MECHANICS
Python Environment Setup

Attilio Frangi, Luca Rosafalco, Matteo Torzoni
Department of Civil and Environmental Engineering

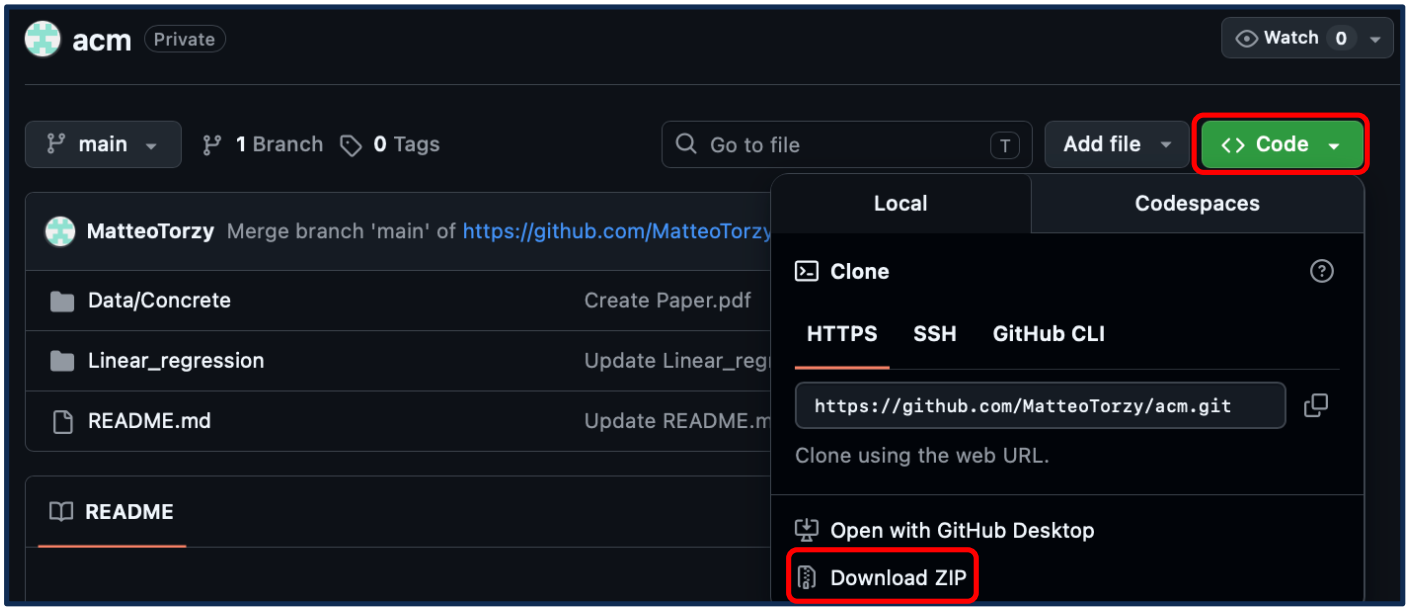
Python: origins, diffusion, and packages

- Python was created at the end of the 1980s, and the first release arrived in 1991. The language was designed with a clear philosophy: code should be simple and intuitive.
- Today, Python is one of the most widely used programming languages in the world. It is extensively adopted in scientific computing, machine learning, automation, and engineering simulations.
- Python is an interpreted language, meaning that code is executed line by line by the Python interpreter rather than being compiled directly into machine code. Python scripts are usually .py files that can be executed from a terminal or an IDE such as Spyder.
- A key reason for its success is the strong open-source community that continuously develops libraries (packages), making advanced functionalities accessible with only a few lines of code.
- Packages are collections of pre-written modules and functions. They can be installed through package managers such as pip or Conda. Once installed, they are imported into a script using a import statement, allowing users to immediately access advanced functionalities.



Download the Advanced Computational Mechanics repository

- Go to the repository page of the course: https://github.com/MatteoTorzy/acm
- Click the green “Code” button and choose “Download ZIP”.



- After downloading the folder, move it to the desired location.

Anaconda distribution

- Anaconda is an open-source Python distribution designed for scientific computing and data science. Rather than installing Python alone, Anaconda provides an ecosystem that includes the Python interpreter, precompiled scientific libraries (such as NumPy, SciPy, pandas, and Matplotlib), and tools for package and environment management.

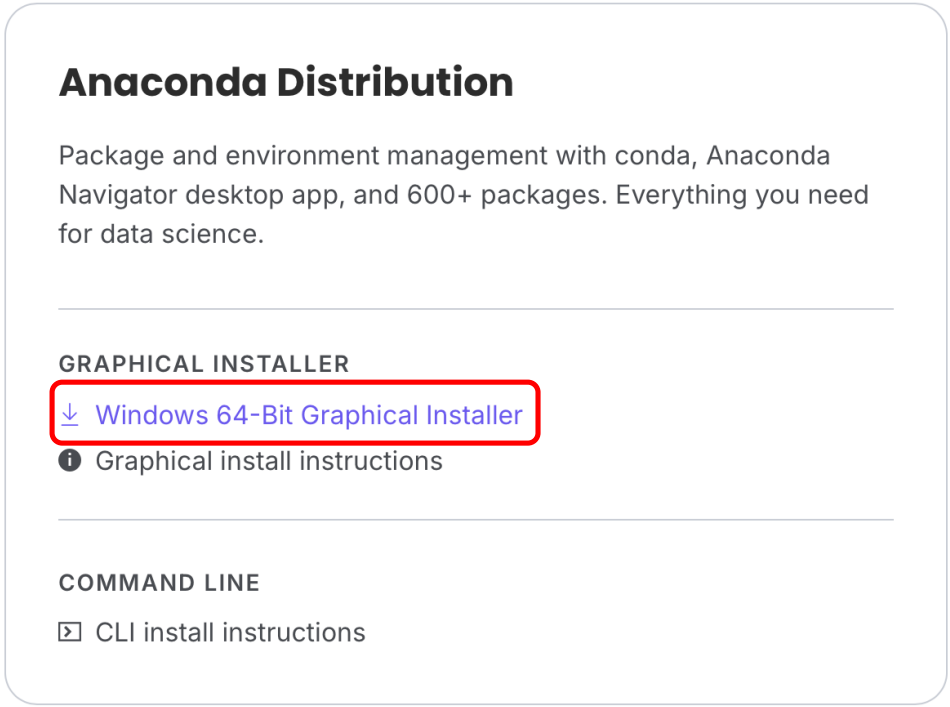
- A key feature of Anaconda is the possibility of creating isolated virtual environments. A virtual environment is a self-contained folder that acts as a standalone workspace. Inside this isolated space, you can install a specific version of Python and a custom set of packages without affecting the rest of your projects. For instance, if project A requires an older version of a library while project B requires a newer one, virtual environments allow both projects to coexist on the same machine.

- To manage virtual environments Anaconda includes the Conda package and environment manager. Unlike the Python standard package manager (pip), Conda can also install non-Python software and compiled binaries dependencies alongside Python libraries.



Download and install Anaconda

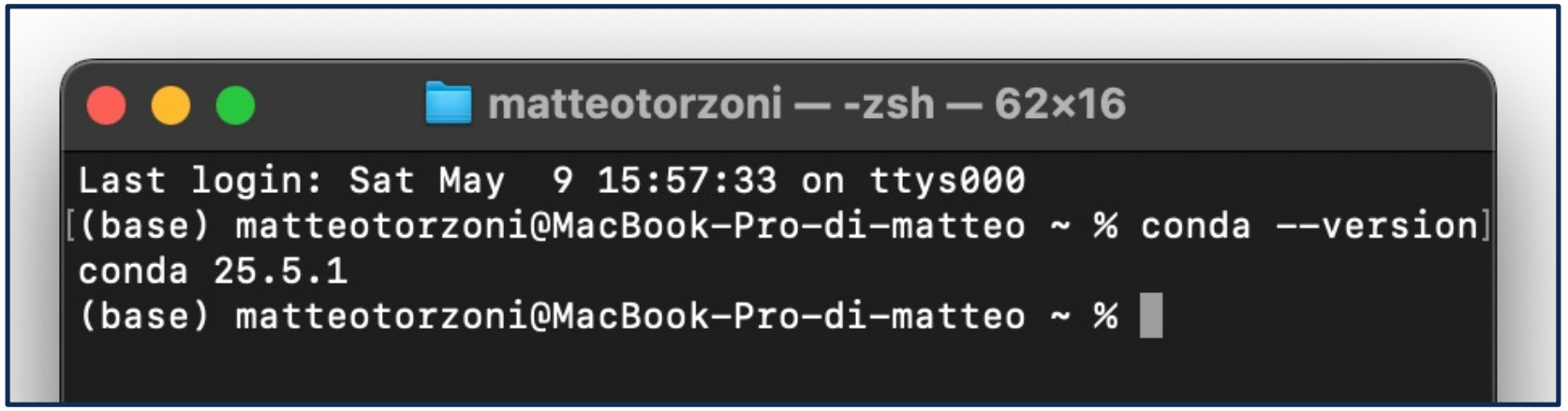
- Go to the Anaconda website: if asked, you can skip the registration as it is not required.
- Download the installer for your operating system (Windows, macOS, or Linux).
- Run the installer and follow the default installation steps.



Verify the installation

- After installation, open Anaconda Prompt (Windows) or Terminal (macOS/Linux).
- You can verify the installation by running the following command to display the installed Conda version:

```
conda --version
```



Create a Conda virtual environment

- Create a new virtual environment with a desired Python version:

```
conda create -n <myenv> python=3.12
```

- Replace <myenv> with your environment name --> “acm”

```
(base) matteotorzoni@MacBookProdimatteo ~ % conda create -n acm python=3.12

2 channel Terms of Service accepted
Retrieving notices: done
Channels:
 - defaults
Platform: osx-arm64
Collecting package metadata (repodata.json): done
Solving environment: done
```

- Enter “y” when Conda asks for confirmation.

```
Proceed ([y]/n)? y


Downloading and Extracting Packages:


Preparing transaction: done

Verifying transaction: done

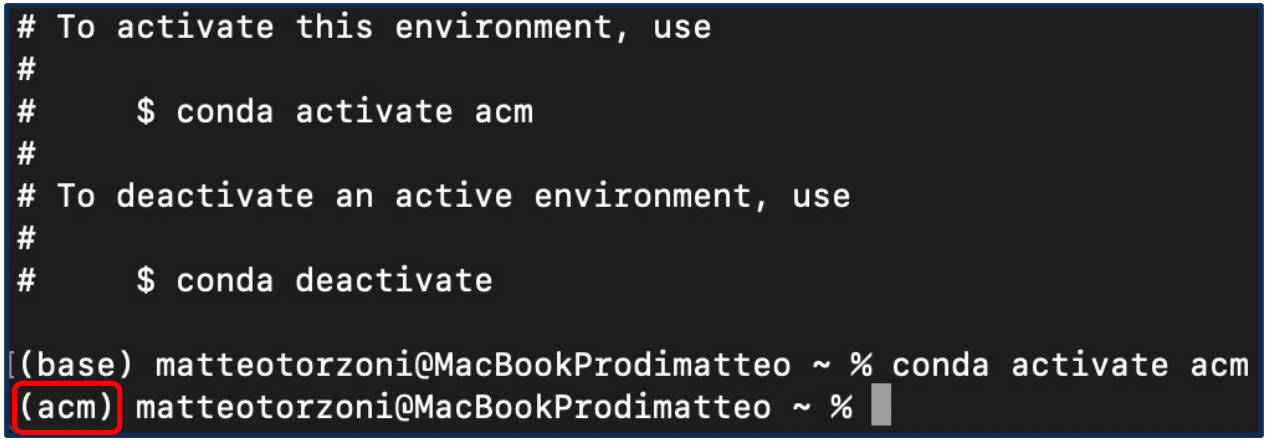
Executing transaction: done
```

Activate the environment

- Activate the new environment before installing packages:

```
conda activate acm
```

- The terminal prompt should now show the environment name:



Jupyter Notebooks: interactive computational documents

- Jupyter Notebook is an interactive environment that allows users to create and execute computational documents in a web browser. A notebook combines Python code, formatted text, mathematical equations, images, and plots within the same document.
- The main advantage of Jupyter Notebooks is their interactivity. Code is organized into cells that can be executed independently, enabling users to test small sections of code step by step. Results such as numerical outputs, tables, and plots are displayed immediately below each cell.
- Notebooks integrate with Python scientific libraries and are compatible with Conda environments.



Install Jupyter Notebook

- Inside the (current) activated environment, install Jupyter Notebook:

```
conda install notebook
```

```
(acm) matteotorzoni@MacBookProdimatteo ~ % conda install notebook
Proceed ([y]/n)? y


Downloading and Extracting Packages:

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
(acm) matteotorzoni@MacBookProdimatteo ~ %
```

- Note: more in general, required packages can be installed with a single command line, without needing to repeat the `conda install` command for each package:

```
conda install numpy matplotlib pandas xlrd scikit-learn
```

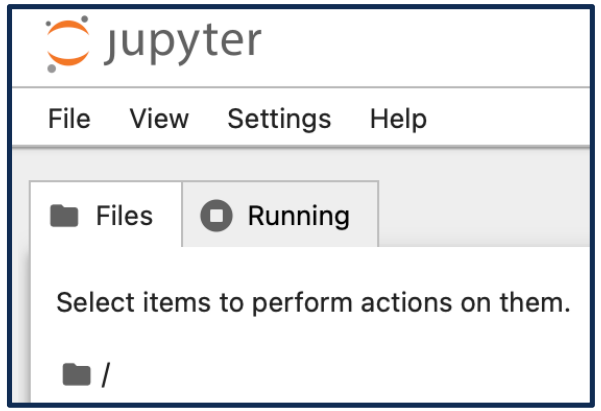
```
(acm) matteotorzoni@MacBookProdimatteo ~ % conda install numpy matplotlib pandas xlrd scikit-learn
  added / updated specs:
    - matplotlib
    - numpy
    - pandas
    - scikit-learn
    - xlrd
```

Open Jupyter notebooks

- Launch Jupyter Notebook using:

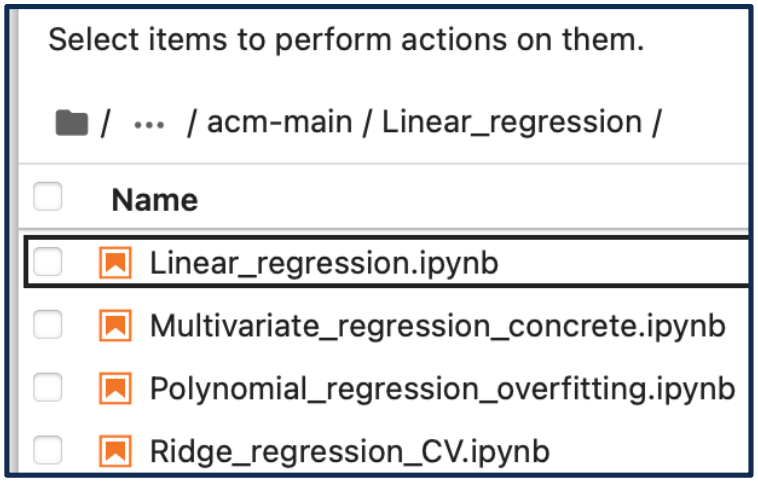
```
jupyter notebook
```

- Jupyter Notebook should open automatically in your browser. →



- Note: by default, it will open in your user home directory (C:\Users\username on Windows or /Users/username on macOS), or in the specific terminal folder where you ran `jupyter notebook`.

- With the explorer, you can navigate to all levels of the children directories to find the location of the `acm` folder containing your Jupyter notebooks.



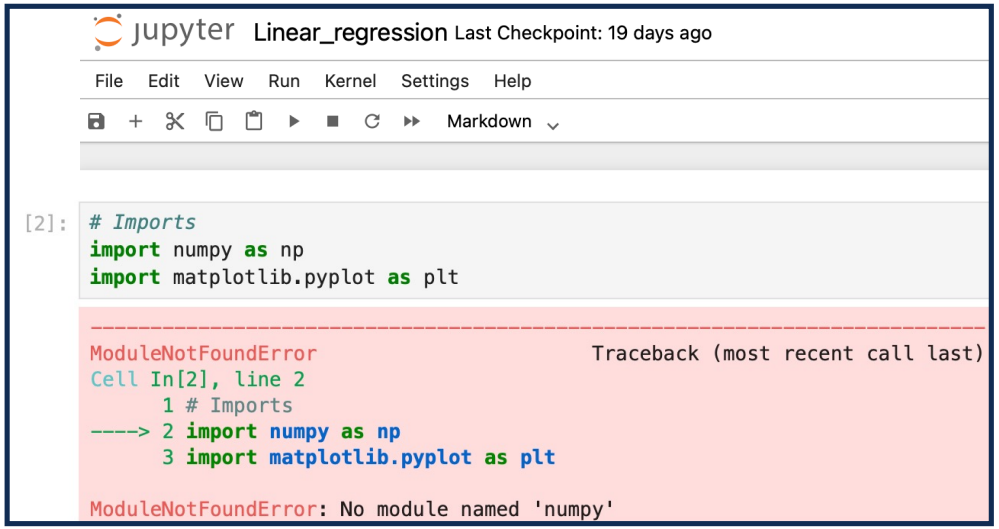
- It is also possible to set the root directory when starting Jupyter. For instance, to set the root at *D:/my_work/jupyter_ipy*:

```
jupyter notebook --notebook-dir D:/my_work/jupyter_ipy
```

- The selected .ipynb notebook file will open in your browser.

Run Jupyter notebooks

- Run notebook cells using Shift + Enter.



- If a package has not been previously installed, it can be done as usual: `conda install numpy matplotlib`

- Note: while Jupyter is running, the Anaconda Prompt (Windows) or Terminal (macOS/Linux) is occupied by the Jupyter server. To install additional packages, you must first stop the Jupyter server by pressing `Ctrl + C` in the Prompt/Terminal and then entering `y` when prompted (closing the browser tab alone is not enough).

```
Shut down this Jupyter server (y/[n])? y
[C 2026-07-25 12:20:24.142 ServerApp] Shutdown confirmed
[I 2026-07-25 12:20:24.146 ServerApp] Shutting down 5 extensions
(acm) matteotorzoni@MacBookProdimatteo ~ %
```

- Alternatively, you can open a new Prompt/Terminal window in parallel, activate the environment, and then run the installation command.

Spyder: scientific Python development environment

- Spyder is an open-source Integrated Development Environment (IDE) for scientific programming with Python. It combines a text editor with tools for code execution and debugging.
- Spyder features an interface similar to MATLAB, which makes it easy to adopt for users coming from engineering backgrounds. The IDE includes a code editor, an integrated console for interactive execution, a variable explorer to inspect data structures in memory, and plotting tools for visualization.
- When installed inside a Conda environment, Spyder can directly access all the packages available in that environment, allowing users to work with libraries such as NumPy, SciPy, and Matplotlib.



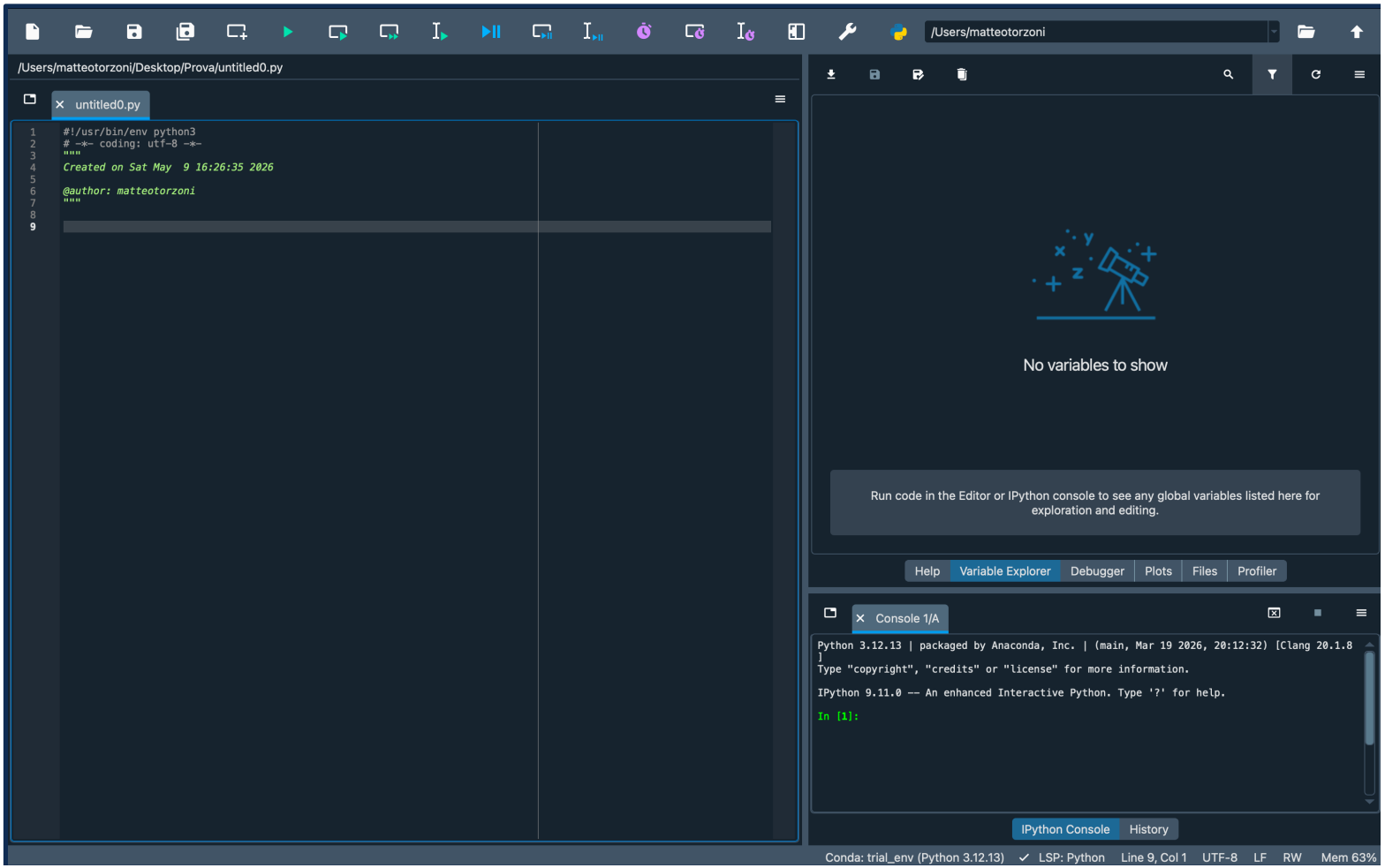
Install Spyder IDE

- Install the Spyder IDE inside the environment:

  ```
  conda install spyder
  ```

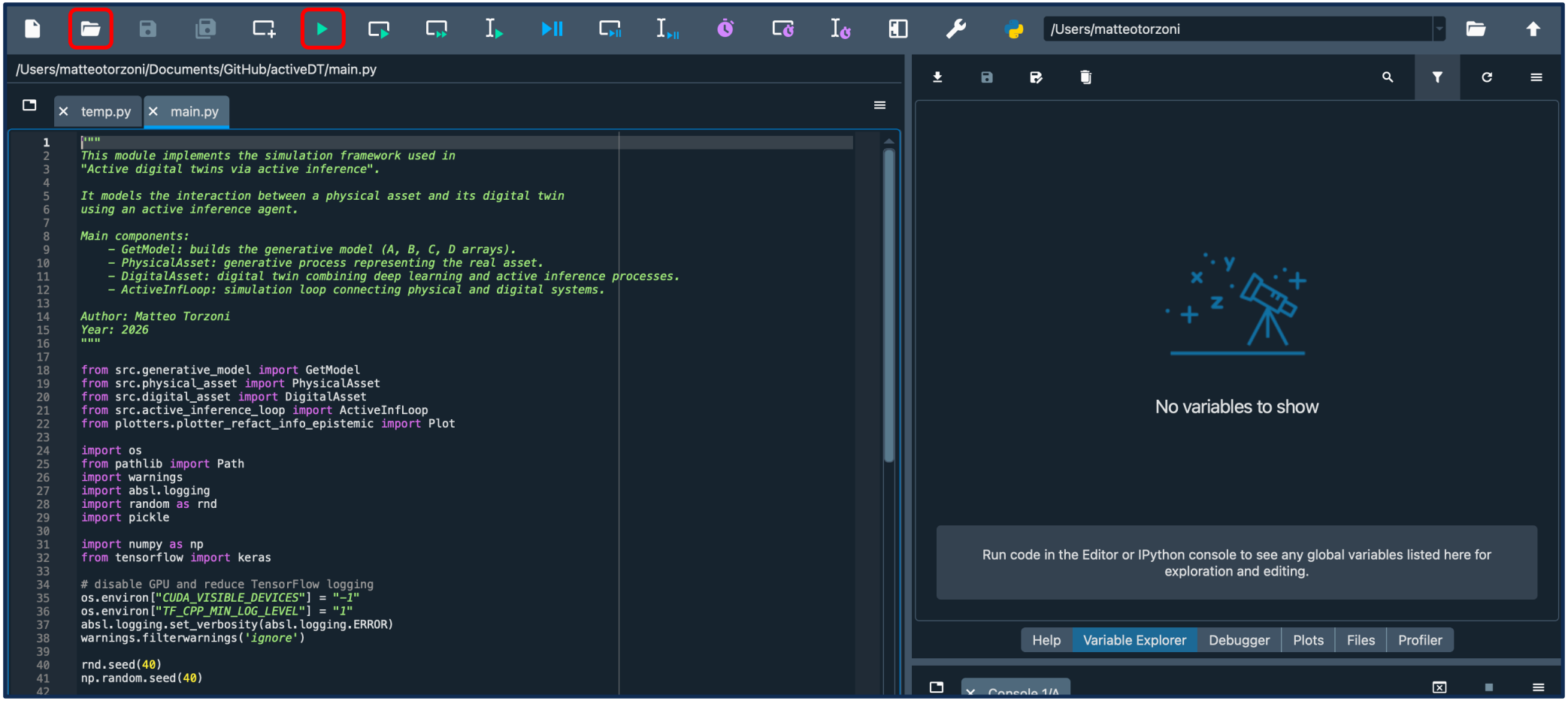
- Lunch Spyder using:

  ```
  spyder
  ```



Run Python scripts

- Open scripts directly in spyder to edit and run them.



Using pip within Conda environments

- Although Conda remains widely used, many Python packages (including PyTorch) are now distributed only through pip. It is the standard Python package manager, used to download and install packages from the Python Package Index (PyPI).

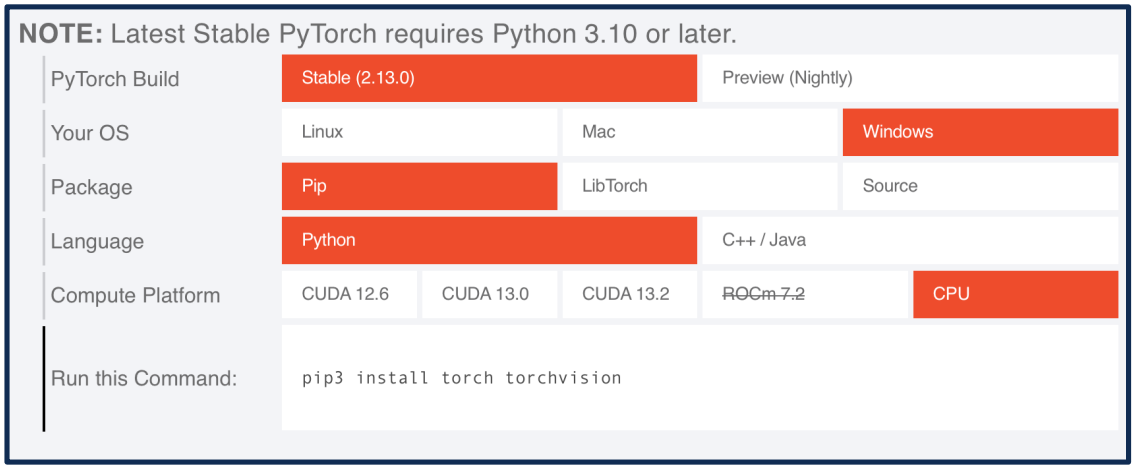


- Consequently, a common workflow is to use Conda to create and manage environments, while installing packages using pip within those environments. This combines the strengths of both: robust environment management from Conda and access to the latest supported package releases via pip. Packages can be installed within the active Conda environment using `pip install`:

```
conda activate acm
pip install torch
```

- The PyTorch page https://pytorch.org provides an interactive support to find the right command for installation, depending on your OS and the computing platform (CPU or GPU).

- Stable versions represent the most currently tested and supported version of PyTorch.



Useful tips (1)

- List all Conda environments:
  `conda env list`
- List all packages installed in the current (activated) environment:
  `conda list`
- Deactivate the current environment:
  `conda deactivate`
- Install specific versions of a package:
  `conda install <pkg>=<version>`
- Install additional packages using pip (if package unavailable in Conda)
  `pip install <package_name>`
- Delete a virtual environment:
  `conda remove --name <myenv> --all`
- Clone a virtual environment:
  `conda create --name <newenv> --clone <oldenv>`

Useful tips (2)

- Prefer `python -m pip` over `pip`. This explicitly uses the `pip` associated with the currently active Python environment, avoiding problems caused by multiple Python installations on the computer:
  ```
  python -m pip install torch
  ```

- Rule of thumb: use `conda` whenever possible and `pip` only when necessary (preferably after installing the conda packages). Conda manages packages together with their dependencies and can resolve compatibility issues more reliably. Using `pip` only after installing the packages available through `conda` reduces the risk of conflicts between the two package managers.

- If a package does not install, try a different version. Compatibility can depend on the Python version and on the versions of other installed packages. A package may work correctly with one version but not another:
  ```
  conda install <pkg>=<version>
  python -m pip install <pkg>==<version>
  ```

- You can also try upgrading or downgrading the package. A recent package update may introduce compatibility issues with other packages already installed in the environment:
  ```
  conda update <pkg>
  python -m pip install --upgrade <pkg>
  ```

- If `conda` has compatibility issues, try the `conda-forge` distribution channel. Different channels may provide different package builds or dependency versions, effectively resolving an installation problem:
  ```
  conda install -c conda-forge <pkg>
  ```

Packages required for the ACM course

- `notebook`
- `numpy`
- `matplotlib`
- `pandas`
- `xlrd`
- `scikit-learn (sklearn)`
- `spyder`
- `torch` (PyTorch)
- `catboost`

- All required packages can be installed with a single command, without needing to repeat the `install` command for each package:

```
conda install notebook numpy matplotlib pandas xlrd scikit-learn spyder catboost

pip install torch
```

GPU support (NVIDIA CUDA)

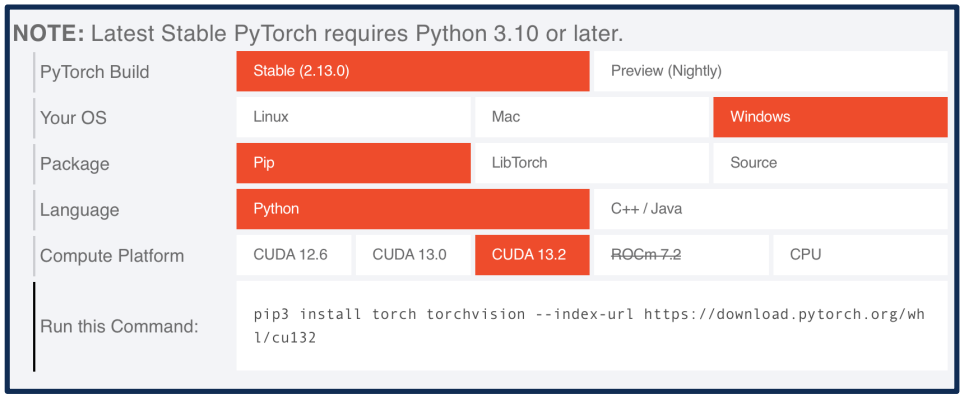
- Create and activate a new virtual environment:

```
conda create -n acmcuda python=3.12
conda activate acmcuda
```

- Install the CUDA-enabled version of PyTorch (assuming an NVIDIA driver 12.4 is already installed):

```
pip install torch --index-url https://download.pytorch.org/whl/cu124
```

- Note: the appropriate CUDA-pytorch version depends on the NVIDIA version supported by your GPU. If your GPU already functions, a compatible NVIDIA driver should already be installed in the machine. Check the driver version and use the correct CUDA-PyTorch installation line from the PyTorch page https://pytorch.org



- Verify that PyTorch detects the GPU (in Spyder console):

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```

Have a nice course!